Experiment 7: Perceptron and Forward Propagation

Question

Problem Statement: A bank wants a quick rule-based screener that approves or rejects small loan applications from two scores. 100 past applications with credit score and income-to-debt ratio, each labelled Approved or Rejected. A second dataset follows the XOR pattern. The objective is to implement a single perceptron with forward propagation and the perceptron learning rule. Use NumPy, Matplotlib.

Tasks:

1. Implement weighted sum, bias and step activation in NumPy.
2. Train the perceptron on the loan data and record weight updates per epoch.
3. Plot the learned decision boundary.
4. Train on the XOR data and show that it fails to converge.
5. Compare step, sigmoid and ReLU outputs for the same inputs.

Code

```
"""Experiment 7: Perceptron and Forward Propagation"""
import numpy as np
import matplotlib.pyplot as plt


np.random.seed(1)


# ---------------- Data 1: loan applications (credit score, income-to-debt ratio) ----------------
credit = np.random.uniform(300, 900, 400)
ratio = np.random.uniform(0.5, 5, 400)
score = 0.6 * (credit - 300) / 600 + 0.4 * (ratio - 0.5) / 4.5
keep = np.abs(score - 0.5) > 0.04                  # keep a clear margin so the data is separable
credit, ratio, score = credit[keep][:100], ratio[keep][:100], score[keep][:100]
y_loan = (score > 0.5).astype(int)                 # 1 = Approved, 0 = Rejected
X_loan = np.column_stack([(credit - 300) / 600, (ratio - 0.5) / 4.5])   # scale to 0-1
print("Loan data:", X_loan.shape, "| approved:", y_loan.sum(), "| rejected:", (1 - y_loan).sum())


# ---------------- Data 2: XOR pattern (100 noisy points) ----------------
base = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
base_y = np.array([0, 1, 1, 0])
X_xor = np.repeat(base, 25, axis=0) + np.random.normal(0, 0.05, (100, 2))
y_xor = np.repeat(base_y, 25)


# ---------------- Task 1: perceptron (weighted sum + bias + step) ----------------
def step(z):
    return (z >= 0).astype(int)


class Perceptron:
    def __init__(self, n_inputs, lr=0.1):
        self.w = np.zeros(n_inputs)
        self.b = 0.0
        self.lr = lr


    def forward(self, X):                          # forward propagation
        z = X @ self.w + self.b                    # weighted sum + bias
        return step(z)                             # step activation


    def fit(self, X, y, epochs=50):
        self.history, self.errors = [], []
        for ep in range(1, epochs + 1):
            errs = 0
            for xi, ti in zip(X, y):
                pred = int(xi @ self.w + self.b >= 0)
                delta = self.lr * (ti - pred)      # perceptron learning rule
                if delta != 0:
                    self.w = self.w + delta * xi
                    self.b = self.b + delta
                    errs += 1
            self.errors.append(errs)
            self.history.append((ep, self.w.copy(), self.b))
```

```
            if errs == 0:
                break

# ---------------- Task 2: train on loan data ----------------
p = Perceptron(2, lr=0.1)
p.fit(X_loan, y_loan, epochs=100)
print("\nEpoch | w1      w2      bias    | misclassified")
for (ep, w, b), e in zip(p.history, p.errors):
    print(f"{ep:5d} | {w[0]:7.3f} {w[1]:7.3f} {b:7.3f} | {e}")
acc = (p.forward(X_loan) == y_loan).mean()
print(f"\nConverged in {len(p.errors)} epochs, training accuracy = {acc*100:.1f}%")

# ---------------- Task 3: decision boundary ----------------
def plot_boundary(model, X, y, title, ax):
    ax.scatter(X[y == 1, 0], X[y == 1, 1], c="green", label="Approved / 1")
    ax.scatter(X[y == 0, 0], X[y == 0, 1], c="red", label="Rejected / 0")
    xs = np.linspace(X[:, 0].min() - 0.1, X[:, 0].max() + 0.1, 100)
    if abs(model.w[1]) > 1e-9:
        ax.plot(xs, -(model.w[0] * xs + model.b) / model.w[1], "k-", lw=2, label="boundary")   # w1x1 + w2x2
+ b = 0
    ax.set_ylim(X[:, 1].min() - 0.1, X[:, 1].max() + 0.1)
    ax.set_xlabel("x1"); ax.set_ylabel("x2"); ax.set_title(title); ax.legend()

fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
plot_boundary(p, X_loan, y_loan, "Loan data: learned decision boundary", ax[0])
ax[1].plot(range(1, len(p.errors) + 1), p.errors, "o-")
ax[1].set_xlabel("Epoch"); ax[1].set_ylabel("Misclassified samples"); ax[1].set_title("Loan data: errors per
epoch")
plt.tight_layout(); plt.show()

# ---------------- Task 4: XOR fails ----------------
px = Perceptron(2, lr=0.1)
px.fit(X_xor, y_xor, epochs=100)
print(f"\nXOR: errors in last 5 epochs = {px.errors[-5:]}  (never reaches 0)")
print(f"XOR accuracy = {(px.forward(X_xor) == y_xor).mean()*100:.1f}%")

fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
plot_boundary(px, X_xor, y_xor, "XOR: a single line cannot separate the classes", ax[0])
ax[1].plot(range(1, len(px.errors) + 1), px.errors)
ax[1].set_xlabel("Epoch"); ax[1].set_ylabel("Misclassified samples"); ax[1].set_title("XOR: errors per epoch
(no convergence)")
plt.tight_layout(); plt.show()

# ---------------- Task 5: step vs sigmoid vs ReLU ----------------
sigmoid = lambda z: 1 / (1 + np.exp(-z))
relu = lambda z: np.maximum(0, z)
z = np.array([-3, -1, -0.5, 0, 0.5, 1, 3])
print("\n   z   |  step | sigmoid |  ReLU")
for zi in z:
    print(f"{zi:6.1f} | {int(zi >= 0):5d} | {sigmoid(zi):7.3f} | {relu(zi):5.2f}")

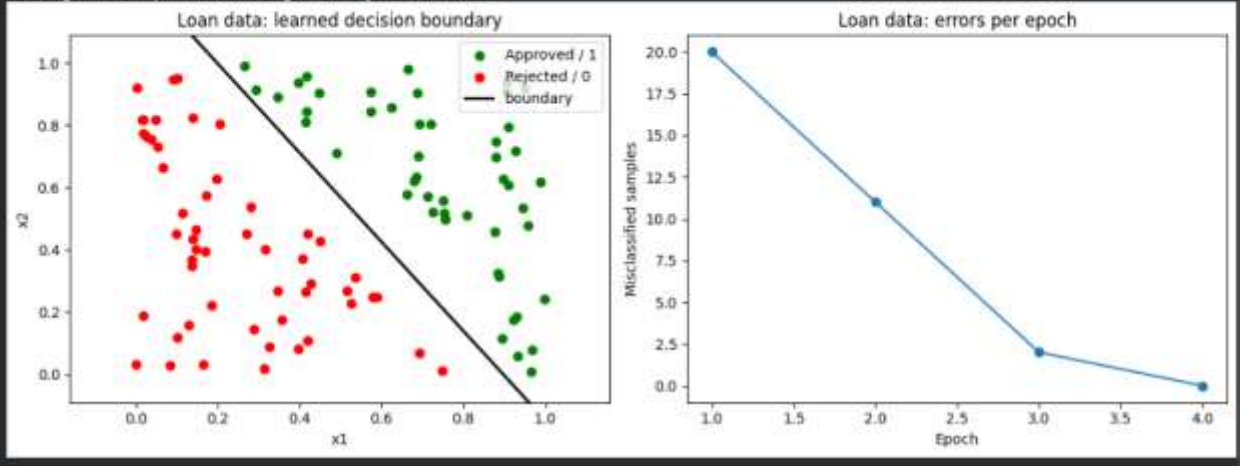
zz = np.linspace(-5, 5, 400)
plt.figure(figsize=(7, 4.5))
plt.plot(zz, step(zz), label="Step"); plt.plot(zz, sigmoid(zz), label="Sigmoid"); plt.plot(zz, relu(zz),
label="ReLU")
plt.ylim(-0.5, 2); plt.xlabel("z"); plt.ylabel("activation(z)"); plt.legend(); plt.grid(alpha=0.3)
plt.title("Activation functions"); plt.show()
```

Output

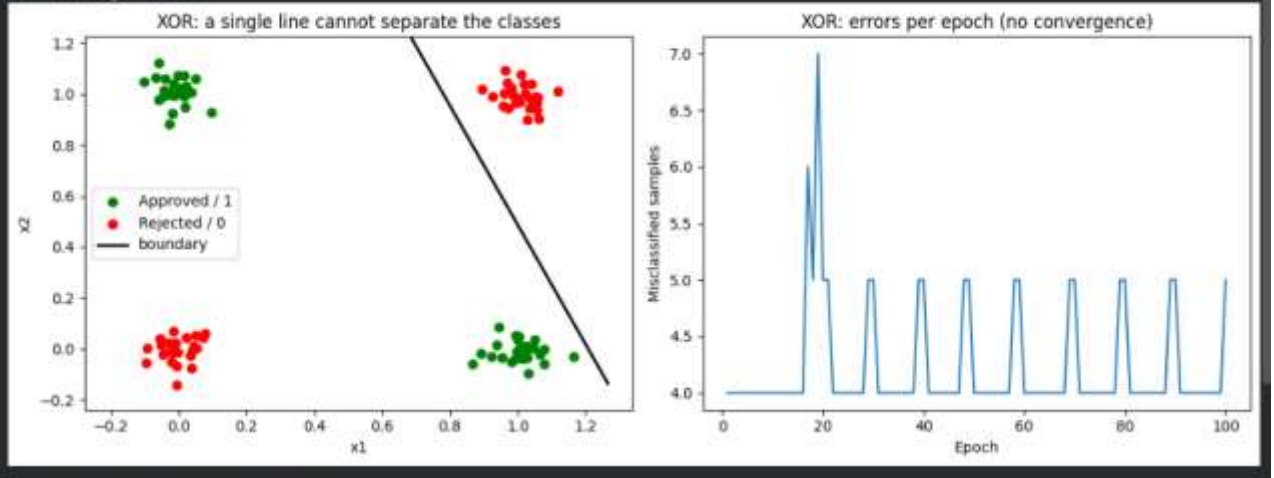
```
Loan data: (100, 2) | approved: 48 | rejected: 52

Epoch | w1      w2      bias    | misclassified
    1 |   0.280   0.105  -0.200 | 20
    2 |   0.310   0.215  -0.300 | 11
    3 |   0.334   0.233  -0.300 | 2
    4 |   0.334   0.233  -0.300 | 0

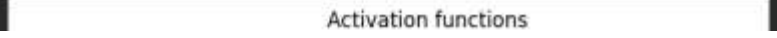
Converged in 4 epochs, training accuracy = 100.0%
```




```
XOR: errors in last 5 epochs = [4, 4, 4, 4, 5]  (never reaches 0)
XOR accuracy = 75.0%
```




```
   z    |  step | sigmoid |  ReLU
 -3.0 |     0 |   0.047 |  0.00
 -1.0 |     0 |   0.269 |  0.00
 -0.5 |     0 |   0.378 |  0.00
  0.0 |     1 |   0.500 |  0.00
  0.5 |     1 |   0.622 |  0.50
  1.0 |     1 |   0.731 |  1.00
  3.0 |     1 |   0.953 |  3.00
```



```
     z   |  step | sigmoid |  ReLU
  -3.0 |     0 |   0.047 |  0.00
  -1.0 |     0 |   0.269 |  0.00
  -0.5 |     0 |   0.378 |  0.00
   0.0 |     1 |   0.500 |  0.00
   0.5 |     1 |   0.622 |  0.50
   1.0 |     1 |   0.731 |  1.00
   3.0 |     1 |   0.953 |  3.00
```

